Tech Lead Assessment

Candidate Brief

Item	Details
Role	Tech Lead
Assessment duration	24 hours
Expected completion effort	Approximately 6–10 hours of actual work
Submission	GitHub repository link
Topic	Industrial equipment monitoring backend service
Assessment format	Take-home backend build

1. Problem Statement

You are building a small backend service for an industrial equipment monitoring platform used in a manufacturing environment.

The platform receives machine-related data from multiple external vendors. These vendors provide machine event, sensor, alert, inspection, and maintenance data through inconsistent schemas and formats. Your task is to design and implement a practical backend system that can: ingest vendor data, normalize it into a usable internal representation, process it asynchronously, produce a derived machine-level operational view, and expose that output through clean backend-facing APIs.

This assessment is designed to evaluate how you think about backend systems, real-world data handling, async workflows, API design, production awareness, and technical trade-offs.

All data and APIs provided are dummy samples created only for this assessment.

2. Files Provided

File / Asset	Purpose
README_FOR_ASSESSMENT_ASSETS.md	Short guide to the assessment asset package
expected_output_context.md	Business context for the downstream derived output
event_type_reference.md	Basic context for vendor event and alert types

asset_reference.csv	Machine and plant reference file
line_reference.csv	Line and plant reference file
vendor_api_samples/	JSON vendor payloads and edge-case batches
seed_data/	CSV seed data for local testing and ingestion

3. What You Need to Build

A. Ingestion Layer

- Provide APIs to accept vendor data from at least two external vendor formats.
- Your ingestion layer should reasonably address request validation, malformed inputs, duplicate handling, versioning assumptions, and basic protection/authentication if you choose to include it.

B. Normalization Layer

- Normalize incoming vendor data into a canonical internal schema.
- Your solution should reasonably address field mapping, machine/entity resolution, enum normalization, timestamp normalization, unit normalization, missing values, invalid values, raw payload traceability, and idempotency assumptions.

C. Async / Background Processing

- A meaningful part of the workflow should be asynchronous.
- Examples include normalization job execution, state recomputation, retryable processing, dead-letter/error handling, and status tracking for accepted payloads.
- You do not need to build a complex distributed system. A simple and well-reasoned approach is preferred.

D. Output Serving Layer

- Expose backend-facing APIs that return ingestion/processing status, machine-level derived operational view, and plant/line-level summary.
- The output should be usable by downstream backend/product teams and should be explainable.

4. Derived Output Context

Your system should produce a machine operational attention view.

- Current derived operational status
- Whether the machine needs attention
- Severity / attention level
- Key contributing reasons or reason codes
- The latest relevant processed signals
- Processing freshness or status context, if you choose to include it

You do not need to implement an ML model. Deterministic, explainable logic is preferred.

5. Suggested Endpoint Categories

You may design your own API structure. At minimum, your system should include equivalents of the following:

- vendor ingestion endpoint(s)
- processing status endpoint
- machine operational view endpoint
- plant / line summary endpoint

You may add other endpoints if they improve clarity.

6. Deliverables

Submit a GitHub repository containing:

Required

1. Source code
2. README including setup instructions, how to run locally, how to test, architecture overview, assumptions, trade-offs, limitations, and what you would do next for production
3. API contract - OpenAPI, Postman collection, or clearly documented endpoints
4. Tests - at least some meaningful tests
5. Local run setup - Docker Compose is preferred, or provide very clear local setup instructions
6. Visible commit history - keep commit history visible; it will be reviewed qualitatively
7. Simple architecture diagram
8. Short design note

7. Technology Guidance

Proffered Stack (Go ? Python or .Net / C#). Preferred qualities are clarity, good engineering judgment, production-oriented choices, and ease of review.

A simple, well-structured solution is better than an overbuilt one.

8. What Not To Do

- Do not build a large frontend.
- Do not over-engineer the assignment with unnecessary infrastructure.
- Do not assume vendor data is always clean, unique, ordered, or complete.
- Do not ignore duplicates, malformed payloads, or conflicting updates.
- Do not return only raw normalized data; expose a useful derived machine-level output.
- Do not skip documentation.
- Do not submit work you cannot explain in discussion.

9. High-Level Evaluation Areas

- System design and architecture judgment
- Backend engineering quality
- Normalization and messy-data handling
- Async/background processing design
- API clarity
- Production awareness
- Documentation and communication
- Testing discipline
- Ease of running and reviewing your project

10. AI Usage Disclosure

You may use AI tools. If you do, disclose briefly which tools were used, how they were used, and what decisions were your own.

You must be able to explain and defend every important part of the submission.